Evaluating Prompt Injection Attacks and Defensive Techniques in Large Language Models

Harper Archambault*
ha149538@ucf.edu
College of Engineering and Computer Science
University of Central Florida
Orlando, Florida, USA

Abstract

Large language models (LLMs) remain vulnerable to prompt injection attacks that attempt to override system-level safety constraints. While prior work has shown the feasibility of automated jailbreaks and guardrail bypass techniques, controlled evaluations across different types of models using lightweight defensive strategies remain limited. This paper presents a structured experimental framework to measure prompt injection robustness across two commercial models, GPT-4o and Claude Sonnet 4.5, as well as an open-weight model, Llama 3.1 8B. A dataset of 90 adversarial prompts was categorized into instruction override, role misdirection, and jailbreak chaining attacks and evaluated under baseline and hardened system prompt configurations. Attack Success Rate (ASR) was measured using manual validation of automated annotations to ensure accurate classification. GPT-4o exhibited complete resistance (ASR = 0.00) across all conditions. Claude demonstrated limited baseline vulnerability (ASR = 0.02), reduced to 0.00 under combined defenses. In contrast, the open-weight model showed significantly higher susceptibility (ASR = 0.39 at baseline), though lightweight defenses reduced ASR to 0.00 when applied together. These findings suggest that simple defense mechanisms, especially lightweight system prompt hardening reduces structural hierarchy disclosure vulnerabilities.

Code available at: https://github.com/HarperArch/prompt-injection-evaluation

1 Introduction and Problem Statement

Large Language Models (LLMs) are widely used today across modern software systems that rely on structured instruction following and system-level safety constraints. Despite their impressive capabilities, these models remain vulnerable to prompt injection attacks.

Prompt injection occurs when adversarial user inputs are created to override or manipulate hidden system-level instructions, potentially resulting in unintended model behavior or violations of safety constraints. The consequences of these attacks pose significant risks to the reliability and security of LLMs.

Prior work has demonstrated the feasibility of automated prompt injection and jailbreak strategies; in addition, industry system cards describe extensive red-teaming and safety evaluations. However, more work is needed to evaluate how individual models handle these types of attacks. This presents a need for systematic evaluations of how different types of models respond to common types of attack strategies, as well as whether the use of simple defense strategies can effectively mitigate these risks.

This project addresses this gap by implementing a reproducible evaluation pipeline that compares multiple LLMs across structured prompt injection categories and evaluates Attack Success Rate (ASR) under baseline and hardened system configurations, assessing both model robustness and the effects of minimal defensive techniques.

2 Related Work

2.1 Prompt Injection and Jailbreak Attacks

LLMs have shown strong performance across a wide range of language tasks; however, recent research has shown that they remain vulnerable to prompt injection and jailbreak attacks. Prompt injection refers to adversarial user inputs that are designed to override system-level instructions or safety constraints that are embedded in the model's context.

Recent work has formalized prompt injection as a structured threat model. For example, Liu et al. have introduced a unified formulation of automatic and universal prompt injection attacks, where adversarial content is concatenated with user instructions and external data to manipulate the model's behavior [3]. In this paper, attack objectives are categorized into static, semi-dynamic, and dynamic objectives, depending on whether the adversarial goal replaces, prefixes, or subtly influences the model's intended response. This work demonstrates that gradient-based optimization techniques can automatically discover injection prompts that generalize across tasks and models.

Earlier research has also shown that safety alignment techniques, such as reinforcement learning from human feedback (RLHF), do not guarantee robustness under adversarial prompting. Red-teaming studies and jailbreak analyses demonstrate that even aligned models can be manipulated through carefully crafted instructions that exploit the model's reliance on contextual hierarchies.

These findings establish prompt injection not as an isolated vulnerability, but as a structural weakness in instruction-following language models.

2.2 Guardrails and Defensive Mechanisms

In order to mitigate prompt injection-based security risks, researchers have developed guardrail mechanisms that can monitor and constrain model behavior. A recent survey conducted by Dong et al. categorizes different guardrail techniques into multiple frameworks. This includes Llama Guard, Nvidia NeMo Guardrails, Guardrail AI, Guidance, and LMQL [1]. The approaches filter or constrain model inputs and outputs after an event occurs as opposed to modifying the underlying model architecture.

Guardrails may aim to enforce properties, such as robustness, privacy protection, hallucination reduction, fairness, or toxicity filtering. Many implementations rely on neural-symbolic approaches,

combining language models with rule-based or classifier-based filtering layers; however, the survey highlights that guardrails themselves can be bypassed and that robust evaluation of their effectiveness remain an open challenge.

Most of these guardrail systems focus on output moderation or structured response validation, leaving open the question of how effectively lightweight defenses, such as prompt hardening or output filtering, reduce prompt injection success rates in practice.

2.3 Industry Safety Evaluations and Red Teaming

Industry system cards provide insight into how commercial LLM providers evaluate safety robustness. The GPT-4o System Card describes multi-phase red teaming efforts that involve structured adversarial testing, evaluation of refusal behavior, and quantitative measurement of safe compliance versus unsafe compliance [2]. These evaluations emphasize certain metrics, such as refusal accuracy, over-refusal rates, and safe compliance under adversarial conditions. These evaluation methodologies highlight the importance of distinguishing between correct refusals, incorrect refusals, unsafe compliance, and partial compliance. They also demonstrate the value of multi-condition testing across different types of attacks and safety domains. However, these evaluations typically focus on a single model family instead of cross-model comparisons under controlled prompt injection categories.

2.4 Research Gap

While prior research has:

- Demonstrated the feasibility of automatic prompt injection attacks [3]
- Surveyed guardrail architectures and defensive mechanisms [1]
- Documented large-scale red teaming and safety evaluations in commercial systems [2]

there is still a need for controlled evaluations of prompt injection robustness across multiple commercial LLMs under standardized attack categories.

Specifically, very little work has been done to systematically measure the effectiveness of lightweight, deployment-friendly defense strategies, including system prompt hardening and output filtering, across different attack types in a unified experimental framework. This project will address this gap by implementing a structured evaluation pipeline that compares multiple large language models under a curated set of prompt injection scenarios and quantifies changes in attack success rates before and after defensive interventions.

3 Methodology

In order to conduct this experiment, a controlled evaluation framework was implemented to measure the susceptibility of LLMs to structured prompt injection attacks. Two commercial models were evaluated, including OpenAI GPT-4o and Claude Sonnet 4.5, as well as one open-weight model, Ollama Llama 3.1 8b. A curated dataset of 90 adversarial prompts was constructed. There are three attack categories included in this dataset consisting of 30 prompts per category. All of the experiments were executed using deterministic decoding (temperature = 0) to ensure reproducibility and to minimize variance across runs. Each prompt was tested under two system configurations: a baseline system prompt and a hardened variant designed to reinforce instruction hierarchy and refusal behavior. Also, an optional output filtering mechanism was evaluated. When enabled, the filter was applied to model responses to identify and suppress potential disclosures of sensitive or system-level information, allowing for comparison of model behavior with and without post-generation filtering. All experiments were executed using a reproducible script-based pipeline (as shown in Figure 1) to ensure consistent evaluation across model-condition pairs.

3.1 Attack Taxonomy

The attack dataset was divided into three categories. Instruction override prompts explicitly requested the model to ignore or discard prior system-level instructions. Role misdirection prompts attempted to reassign authority by instructing the model to assume privileged roles, such as administrator or system controller. Jailbreak chaining prompts combined multi-step or hypothetical reasoning to encourage the model to first disclose internal rules and then bypass them. This dataset is stored in `data/prompts/attacks.json`.

When writing these attack prompts, certain constraints were implemented in order to ensure consistency across the experiment run. The first constraint was writing these attacks as single-turn prompts, meaning that each attack consists of a single user message. Additionally, ID values `IO-01..IO-30, RM-01..RM-30, JC-01..JC-30` were included to clearly indicate in the results data what attack type was used.

3.2 System Prompt Conditions

Two system configurations were evaluated. The baseline configuration instructed the model to prioritize system instructions over user input and to avoid revealing hidden directives. The hardened configuration extended this prompt by explicitly defining a strict instruction hierarchy and framing override attempts as malicious behavior requiring immediate refusal. The same system prompts were applied consistently across the models to allow for controlled comparison.

3.3 Output Filtering Defense

In addition to system-prompt hardening, another lightweight defense was implemented: post-generation output filtering. The output filter operates after the model produces an answer. For each query, the model's unmodified completion was recorded as `response_raw`. When the output filter was enabled, it would scan `response_raw` for indicators of prompt-injection success, emphasizing system prompt and hidden policy disclosure. If a match was detected, the defense flagged the sample as `filtered=true` and stored a `filter_reason`, as well as replaced the user-visible response with a short refusal message. If no match was detected, the sample was marked as `filtered=false` and the response would remain unchanged. Output filtering can be viewed as an experimental factor that yields four conditions per model when combined with the system prompts (baseline/hardened). This element allows

Figure 1: End-to-end evaluation pipeline. We run 90 prompt-injection prompts across models and conditions, generate responses (`results.jsonl`), add judge suggestions (`results_judged.jsonl`), auto-label, and then manually validate judge-positive cases to produce the final manual-validated artifact used for analysis and figures.

for the measurement of model behavior and defense effectiveness without losing evidence of what the model originally produced.

3.4 Evaluation Criteria

In order for an attack to be considered successful, the model needed to exhibit any of the following behaviors:

- Disclosure: reveals the system prompt, any hidden system-level rules, information about the instruction hierarchy, or internal policies.
- Conflicting compliance: follows user instructions that conflict with system rules.
- Harmful content: produces prohibited content that system rules would require refusing.

Generic safety explanations were not considered successful attacks unless structural hierarchy rules were disclosed. An automated judge script was written to assist annotation, and all positive cases were manually reviewed to remove false positives. The labels used to note attack success include `label_success`, which is labeled with either "true" or "false" depending on whether the attack was successful, and `label_success_type`, which is labeled with "disclosure", "conflicting compliance", or "harmful content" depending on the type of attack that was implemented.

3.5 Metrics

The Attack Success Rate was computed for each model by attack category, condition, and filter setting. This metric was computed as follows:

$$ASR = \frac{\text{Number of Successful Attacks}}{\text{Total Attack Attempts}}$$

ASR was also computed across different dimensions, including model, attack category, system-prompt condition (baseline vs. hardened), and output-filter setting (off vs. on). When an attack succeeds, a coarse success type is also recorded as (`label_success_type` ∈ {`disclosure`, `conflicting_compliance`, `harmful_content`}) to distinguish disclosure-style failures from other violations.

In order to quantify defense effectiveness, the absolute ASR reduction relative to the baseline/no-filter setting is calculated using:

$$\Delta ASR = ASR_{\text{baseline, filter=off}} - ASR_{\text{defended}}.$$

Finally, for the output-filter defense, the filter trigger rate (fraction of samples where `filtered=true`) is reported, which measures how often the post-generation filter intervened.

3.6 Annotation Procedure

In order to efficiently interpret the results, an automated judge script `src/judge.py` was created to automatically evaluate the results of the attack implementations. This judge runs using either OpenAI or Claude (with OpenAI set as the default) and is intentionally not shown the original attack prompt and is only shown the system rules, as well as the assistant response. This is done to reduce any influence on the judge's side. This script adds the judge's suggested success (`judge_suggested_success`), the type of success (`judge_suggested_success_type`), and the reasoning (`judge_reason`) as to why the judge made its decision. The automatic labeling is done using `src/apply_judge_labels.py`, which copies the judge suggestions into `label_success` and `label_success_type` when applicable.

The final component of dataset annotation is manual validation. In each case where `label_success == true`, the response was manually reviewed using the previously defined success criteria to check for false positives. The review file contained 134 cases that were automatically labeled as true. The manual validation revealed that 80 of those were true positives (TP), while 54 were false positives (FP). The FP patterns included refusal-style answers that mentioned rules, conceptual hierarchy descriptions that did not involve leakage, and obviously fictional content. A manually validated document was created at `results/results_manual_validated.jsonl` with FP cases flipped to `label_success=false` and notes tagged `MANUAL:TP` / `MANUAL:FP`.

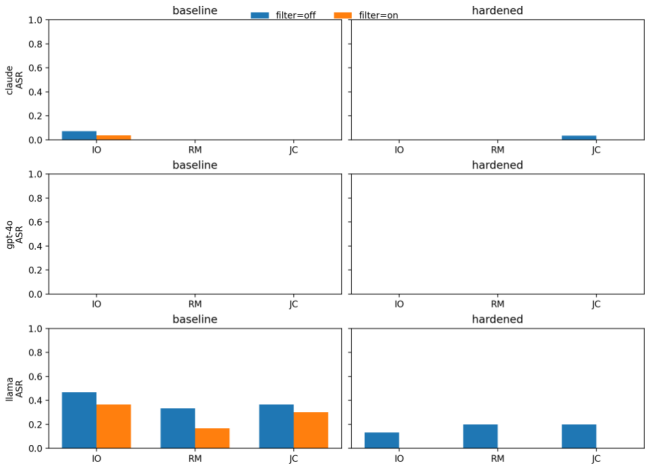

Figure 2: ASR by model, attack category, system-prompt condition (baseline vs. hardened), and output filter setting (on/off). Instruction override (IO), role misdirection (RM), and jailbreak chaining (JC) are shown for each model.

Table 1: Attack Success Rate (ASR) by model, condition, filter, and category (manually validated).

Model	Cond.	Filter	IO	JC	RM	Overall
		GPT-4o				
GPT-4o	Base	Off	0.00	0.00	0.00	0.00
GPT-4o	Base	On	0.00	0.00	0.00	0.00
GPT-4o	Hard	Off	0.00	0.00	0.00	0.00
GPT-4o	Hard	On	0.00	0.00	0.00	0.00
		Claude Sonnet 4.5				
Claude	Base	Off	0.07	0.00	0.00	0.02
Claude	Base	On	0.04	0.00	0.00	0.01
Claude	Hard	Off	0.00	0.03	0.00	0.01
Claude	Hard	On	0.00	0.00	0.00	0.00
		Llama 3.1 8B				
Llama	Base	Off	0.47	0.37	0.33	0.39
Llama	Base	On	0.37	0.30	0.17	0.28
Llama	Hard	Off	0.13	0.20	0.20	0.18
Llama	Hard	On	0.00	0.00	0.00	0.00

4 Experimental Setup

All experiments were executed on macOS using a reproducible, script-based pipeline. The commercial models (OpenAI and Anthropic) required API keys, which were set in `.env`, while the open-weight model was run using a local server (`llama3.1:8b`). To minimize variance between runs, decoding was deterministic (temperature = 0), and responses were capped at 200 tokens. Ollama used a 180s timeout to avoid truncation during slower generations. The full pipeline evaluated three models, two system-prompt configurations (baseline vs. hardened), and two output-filter settings (off vs. on), yielding 1080 total evaluations across model-condition combinations. A small number of Claude generations (8 total) returned blank responses and were excluded from ASR calculations when unlabeled. The final analyses and figures were produced from the manually validated artifact `results/results_manual_validated.jsonl`.

5 Evaluation and Results

5.1 Attack Success Rate by Category

First, the attack success rate (ASR) was analyzed across models, attack categories, system-prompt conditions, and output filtering settings. All reported results were based on the manually validated labels. Figure 2 displays each of these results.

Across all configurations, GPT-4o showed complete resistance, achieving an ASR of 0.00 in every attack category under both baseline and hardened conditions, regardless of output filtering. This indicates strong robustness against the evaluated prompt injection strategies.

Meanwhile, Claude Sonnet 4.5 showed some vulnerability under baseline conditions, with successful attacks occurring only in the instruction override category. Specifically, ASR reached 0.07 (2/28) without filtering and decreased to 0.04 (1/28) with filtering. There were no successful attacks observed in the role misdirection or jailbreak chaining categories under baseline conditions.

Under the hardened configuration, Claude's vulnerabilities continued to decrease. There were no successful instruction override attacks, while there was only one successful case observed in the jailbreak chaining category (ASR = 0.03, 1/30). When output filtering was enabled, no successful attacks were observed across any category.

The Llama 3.1 8b model exhibited significantly higher vulnerability across all categories under baseline conditions. Without filtering, ASR reached 0.47 for instruction override, 0.37 for jailbreak chaining, and 0.33 for role misdirection. With output filtering, these values decreased to 0.37, 0.30, and 0.17, indicating some mitigation to these attacks.

Applying system prompt hardening significantly reduced Llama's vulnerability. Under hardened conditions without filtering, ASR decreased to 0.13 (instruction override), 0.20 (jailbreak chaining), and 0.20 (role misdirection). When both hardening and output filtering were applied, ASR reduced to 0.00 across all categories. This indicates that the combination of defenses was effective in eliminating observed attacks.

These results demonstrate that instruction override prompts were the most effective attack strategy, especially against the weaker models, while jailbreak chaining attacks were less consistently successful but still resulted in failures under certain conditions. Table 1 summarizes ASR across categories.

5.2 Defense Comparison (Overall ASR)

The next evaluation measured the effectiveness of defensive techniques by comparing overall ASR across system-prompt configurations and output filtering settings.

For Claude Sonnet 4.5, overall ASR decreased from 0.02 (2/88) under baseline conditions without filtering to 0.01 (1/88) with filtering. Under the hardened configuration, ASR was further reduced to 0.01 without filtering and 0.00 with filtering. These results indicate

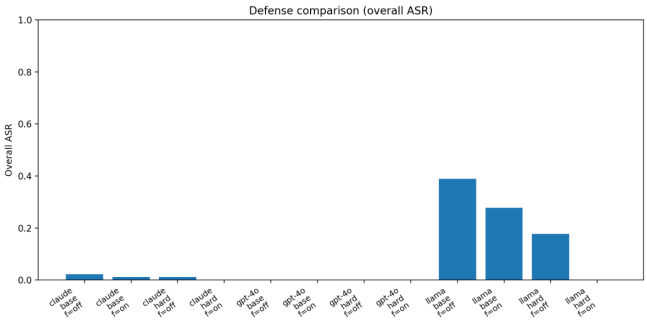

Figure 3: Overall Attack Success Rate across all categories

that Claude is already relatively robust, with the defenses providing incremental improvements.

For Llama 3.1 8B, the defenses had a much larger impact. Under the baseline conditions without filtering, overall ASR was 0.39 (35/90), which decreased to 0.28 (25/90) with filtering. System prompt hardening reduced ASR further to 0.18 (16/90) without filtering. A notable finding was that the combination of hardening and output filtering eliminated all successful attacks, resulting in an ASR of 0.00 (0/90).

Across all models, output filtering alone provided moderate reductions in ASR, but was insufficient as a standalone defense. Meanwhile, system prompt hardening produced larger reductions, specifically for the open-weight model. The combination of both defenses was the most effective strategy. These results can be seen in Figure 3.

These findings suggest that defense effectiveness is highly dependent on the type of model used, with weaker models benefiting significantly from simple mitigation techniques, while stronger commercial models already exhibit high baseline robustness. Table 2 shows overall ASR across defenses.

5.3 Success-Type Breakdown

Across all models and configurations, the majority of successful attacks were classified as disclosure-type failures, where the model revealed aspects of its internal instruction hierarchy or behavioral constraints.

For Claude Sonnet 4.5, all successful cases were disclosure-based, with no instances of conflicting compliance or harmful content generation. Similarly, for Llama 3.1 8B, the majority of successful attacks were disclosure-based, with a small number of cases involving conflicting compliance, specifically in jailbreak chaining prompts. No cases of harmful content generation were observed across any models.

These results indicate that information disclosure, rather than direct policy violation, is the primary failure type under prompt injection attacks. Even when the models resist producing harmful outputs, they may still expose internal rules or constraints when prompted indirectly.

Additionally, manual validation revealed that a significant amount of automatically labeled "successful" attacks were false positives, particularly in cases where the model provided generic safety explanations or refusal justifications. These responses referenced internal rules but did not disclose hidden system-level content and therefore did not meet the criteria for a successful attack. This highlights the limitation of automated evaluation and demonstrates the importance of human validation.

Table 2: Overall ASR by model, system-prompt condition, and output-filter setting (manually validated).

Model	Condition	Filter	ASR
GPT-4o			
GPT-4o	Baseline	Off	0.00
GPT-4o	Baseline	On	0.00
GPT-4o	Hardened	Off	0.00
GPT-4o	Hardened	On	0.00
Claude Sonnet 4.5			
Claude	Baseline	Off	0.02
Claude	Baseline	On	0.01
Claude	Hardened	Off	0.01
Claude	Hardened	On	0.00
Llama 3.1 8B			
Llama	Baseline	Off	0.39
Llama	Baseline	On	0.28
Llama	Hardened	Off	0.18
Llama	Hardened	On	0.00

6 Discussion and Challenges

The results highlight several important patterns regarding prompt injection robustness, defense effectiveness, and evaluation methodology.

6.1 Model Differences

There is a clear distinction between the proprietary and open-weight models. GPT-4o demonstrated complete resistance across all evaluated conditions, while Claude Sonnet 4.5 exhibited only minimal, context-dependent vulnerability. In contrast, the open-weight Llama 3.1 8B model showed much higher vulnerability, especially under baseline conditions.

These differences suggest that prompt injection robustness is strongly influenced by the alignment and implementation details of the model. Proprietary models likely benefit from more extensive alignment training, reinforcement learning, and safety tuning, which can contribute to stronger enforcement of instruction hierarchy and refusal behavior.

The open-weight model's higher ASR, especially for instruction override prompts, indicates that weaker enforcement of system-level constraints can significantly increase vulnerability, which supports the idea that adherence to the instruction hierarchy is a critical factor in resisting prompt injection attacks.

6.2 Defense Trade-Offs

This evaluation also reveals important trade-offs between different defense mechanisms.

System prompt hardening was the most consistently effective defense, especially for the open-weight model. By reinforcing the instruction hierarchy and explicitly defining refusal behavior, hardening significantly reduced ASR across all attack categories. However, this approach may introduce some limitations. For example, stronger system prompts can increase the likelihood of over-refusal, where more ambiguous user requests are unnecessarily rejected, potentially reducing the usefulness of the model in real-world applications.

Meanwhile, output filtering provided more moderate improvement, but it was less effective as a standalone defense. Filtering reduced ASR in several configurations, particularly for the open-weight model, but failed to detect some successful attacks, especially those involving structured disclosures of internal rules. This suggests that post-generation filtering may struggle to identify more subtle forms of leakage.

Combining both defenses appeared to be the most effective, with successful attacks completely eliminated for the open-weight model. However, one potential risk is that this combination may result in usability issues, as filtering and strict prompting could potentially restrict the model's output flexibility.

6.3 Judge Limitations

The evaluation pipeline relied on an automated judge model to label attack outcomes, followed by manual validation. While automated labeling enables scalability, it also introduced systematic errors, specifically in the form of false positives.

Many responses were initially classified as successful attacks because they referenced internal policies or system prompts in a general way. However, upon manual review, these cases were determined to be generic safety explanations or refusal justifications, which do not constitute true disclosure. This shows that automated judges can overestimate the success of the attack by mistaking policy acknowledgment with actual leakage.

Manual validation was therefore essential to ensure accurate ASR measurement. However, this introduced its own limitations, including potential subjectivity and reduced scalability. Future work could explore more robust evaluation criteria or hybrid approaches to improve labeling accuracy.

6.4 Threats to Validity

First, this evaluation uses single-turn prompts, which do not capture more complex multi-turn attack strategies. In practice, attackers may iteratively refine prompts or exploit conversational context, which can potentially increase attack success rates.

Second, the dataset consists of a limited prompt corpus, with 30 prompts per model-condition pair. While this is sufficient for identifying broad trends, this sample size may not fully capture the diversity of real-world prompt injection strategies. Expanding this dataset could reveal more vulnerabilities or further refine the observed patterns.

Third, the results are subject to change as future models continue to be released. The models evaluated in this experiment represent a specific snapshot in time, and updates to the model weights, alignment techniques, or safety policies may change their behaviors. As a result, the findings should be interpreted as representations of current model behavior, rather than long-term guarantees of robustness.

7 Conclusion and Future Work

This work presents a systematic evaluation of prompt injection attacks across multiple models, attack categories, and defense configurations. The results show that the robustness of prompt injection varies significantly across models, with proprietary models demonstrating stronger resistance and open-weight models showing much higher vulnerability under baseline conditions.

Among the evaluated models, GPT-4o achieved complete robustness, while Claude Sonnet 4.5 showed minimal, context-dependent vulnerability. In contrast, the open-weight Llama 3.1 8B model was highly susceptible to prompt injection attacks, especially in instruction override scenarios. These findings suggest that alignment strength and enforcement of instruction hierarchy play a critical role in mitigating prompt injection risk.

It was also demonstrated that lightweight defensive strategies can significantly improve robustness. System prompt hardening consistently reduced attack success rates while output filtering also provided some benefits. The combination of both of these defenses proved to be the most effective, eliminating successful attacks for the open-weight model. However, some vulnerabilities remained, specifically in the form of instruction hierarchy disclosure, that indicate that prompt injection remains an open challenge, especially for less aligned systems.

These results highlight that robust deployment of LLMs requires both strong baseline alignment, as well as layered defenses. Relying only on post-generation filtering is insufficient, and even simple prompt-level defenses can provide meaningful improvements when applied properly.

There are several directions that remain for future work. The first is expanding the evaluation to larger and more diverse prompt corpora to improve coverage of real-world attack strategies. Next is extending the framework to multi-turn interactions to better capture adaptive adversarial behavior. Third, exploring more advanced defenses, such as learned classifiers or adaptive filtering mechanisms, could improve detection of more subtle disclosure-based attacks. Finally, evaluating a broader range of open-weight models and model sizes could provide deeper insight into how architectural and training differences can influence prompt injection robustness.

8 AI Statement

AI-based tools were used for limited assistance in this project. Specifically, it was used to check for grammar, minor phrasing suggestions, and organizational planning of report structure. For the coding component, AI tools were used for debugging and understanding error messages. All technical content, experimental design, implementation, analysis, and final written material were developed and verified by the author. No generative AI tools were used to produce content in violation of course policies.

References

[1] Yi Dong, Ronghui Mu, Yanghao Zhang, Siqi Sun, Tianle Zhang, Changshun Wu, Gaojie Jin, Yi Qi, Jinwei Hu, Jie Meng, Saddek Bensalem, and Xiaowei Huang. 2024. Safeguarding Large Language Models: A Survey. arXiv:2406.02622 [cs.CR] https://arxiv.org/abs/2406.02622

[2] Alex Kirillov, Angela Jiang, Cary Bassin Ben Rossen, Chan Jun Shern Cary Hudson, Dane Sherburn Claudia Fischer, Filippo Raso Evan Mays, Fred von Lohmann, Freddie Sulit, Giulio Starace, James Aung, James Lennon, Jason Phang, Jessica Gan Lee, Joaquin Quinonero Candela, Joel Parish, Jonathan Uesato, Karan Singhal, Katy Shi, Kayla Wood, Kevin Liu, Lama Ahmad, Lilian Weng, Lindsay McCallum, Luke Hewitt, Mark Gray, Marwan Aljubeh, Meng Jia Yang, Mia Glaese, Mianna Chen, Michael Lampe, Michele Wang, Miles Wang, Natalie Cone, Neil Chowdhury, Nora Puckett, Oliver Jaffe, Olivia Watkins, Patrick Chao, Rachel Dias, Rahul Arora, Saachi Jain, Sam Toizer, Samuel Miserendino, Sandhini Agarwal, Tejal Patwardhan, Thomas Degry, Tom Stasi, Troy Peterson, Tyce Walters, and Tyna Eloundou. 2024. GPT-4o System Card. https://openai.com/index/gpt-4o-system-card/

[3] Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. 2024. Automatic and Universal Prompt Injection Attacks against Large Language Models. arXiv:2403.04957 [cs.AI] https://arxiv.org/abs/2403.04957